WEEK 2

Building an LLM Client Using Gemini and Ollama

Student Name	Allamsetti Lahari
Course / Branch	B.Tech – CSIT
Subject / Lab	Gen Ai

Objective

The objective of this lab is to build a provider-agnostic Large Language Model (LLM) client in Python. The application provides one common interface to generate responses using either Google Gemini, a hosted AI service, or Ollama, which runs a language model locally. The user selects a provider, enters a prompt, and receives a response through the same client interface.

Introduction

A Large Language Model (LLM) is an AI model that can understand and generate human-like text. Different LLM providers may use different connection methods. This project brings those methods behind one common generate(prompt) interface.

- Google Gemini: a hosted AI model accessed through the Google Gen AI Python SDK and an API key.
- Ollama: a local model runner. This project uses the llama3.2 model through Ollama's local HTTP API.

Provider-agnostic means that the main application can use the same client method without needing to know the provider's internal connection details.

3. Tools and Technologies

Tool / Technology	Purpose
Python	Programming language used to implement the client.
Visual Studio Code	Code editor used to create and edit project files.
Google Gemini	Hosted LLM provider.
Ollama	Runs the Llama 3.2 model locally.
google-genai	Python SDK used to connect to Gemini.
requests	Sends an HTTP request to the local Ollama API.
python-dotenv	Loads environment variables from the .env file.
unittest	Python's built-in framework used for the unit test.
pytest	Included in requirements.txt; the demonstrated test uses unittest.

Project Setup

A folder named week-2 was created on the Desktop and opened in Visual Studio Code. The following files were created:

```
week-2/
│
├── .env
├── .gitignore
├── llm_client.py
├── main.py
├── test_client.py
└── requirements.txt
```

The .env file stores the Gemini API key locally. The .gitignore file excludes the key file and common generated folders from Git.

5. Installation and Configuration

5.1 Install and verify Ollama

Ollama was downloaded and installed on Windows. Installation was verified in PowerShell with:

```
PS C:\WINDOWS\system32> ollama --version
ollama version is 0.40.16
```

5.2 Download the local model

The Llama 3.2 model was downloaded with:

```
PS C:\WINDOWS\system32> ollama pull llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100%                                                2.0 GB
pulling 966de95ca8a6: 100%                                                1.4 KB
pulling fcc5a6bec9da: 100%                                                7.7 KB
pulling a70ff7e570d9: 100%                                                6.0 KB
pulling 56bb8bd477a5: 100%                                                  96 B
pulling 34bb5ab01051: 100%                                                 561 B
verifying sha256 digest
writing manifest
removing unused layers
success
```

5.3 Install Python dependencies

The following lines were added to requirements.txt:

```
python-dotenv
pytest
google-genai
requests
```

The packages were installed using:

```
python -m pip install -r requirements.txt
```

5.4 Configure the Gemini API key

The same Gemini API key that had worked in Week 1 was reused. It was placed in the .env file in this format:

```
GEMINI_API_KEY=your_actual_gemini_api_key
```

5.5 Configure .gitignore

```
.env
.venv/
__pycache__/
```

6. Implementation

6.1 llm_client.py

This file defines the common provider interface, the Gemini implementation, the Ollama implementation, and the common LLMClient class.

```
from abc import ABC, abstractmethod
from google import genai
import requests



# Step 1: Create a common interface
class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass



# Step 2: Gemini Provider
class GeminiProvider(LLMProvider):

    def __init__(self, api_key):
        self.client = genai.Client(api_key=api_key)

    def generate(self, prompt):
        response = self.client.models.generate_content(
            model="gemini-3.5-flash-lite",
            contents=prompt
        )
        return response.text



# Step 3: Ollama Provider
class OllamaProvider(LLMProvider):

    def generate(self, prompt):
        url = "http://localhost:11434/api/generate"

        data = {
            "model": "llama3.2",
            "prompt": prompt,
            "stream": False
        }

        response = requests.post(url, json=data)
        response.raise_for_status()

        return response.json()["response"]
```

```
# Step 4: Common LLM Client
class LLMClient:

    def __init__(self, provider):
        self.provider = provider

    def generate(self, prompt):
        return self.provider.generate(prompt)
```

6.2 main.py

This is the entry point. It loads environment variables, asks the user to select a provider, creates that provider, and sends the prompt through LLMClient.

```
import os
from dotenv import load_dotenv

from llm_client import LLMClient, GeminiProvider, OllamaProvider



# Load environment variables
load_dotenv()

# Ask the user to select an AI provider
print("Choose your AI Provider:")
print("1. Gemini")
print("2. Ollama")

choice = input("Enter your choice (1 or 2): ")

# Select the provider
if choice == "1":
    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        raise ValueError("Gemini API key not found in .env file!")

    provider = GeminiProvider(api_key)

elif choice == "2":
    provider = OllamaProvider()

else:
    print("Invalid choice!")
    exit()

# Create the common LLM client
client = LLMClient(provider)

# Get prompt from user
prompt = input("\nEnter your prompt: ")

# Generate AI response
response = client.generate(prompt)

print("\nAI Response:")
print(response)
```

6.3 test_client.py

This file tests LLMClient with a fake provider. It checks that the client forwards the prompt and returns the provider's response. The test does not call Gemini or Ollama.

```
import unittest
from llm_client import LLMClient



# Create a fake AI provider for testing
class FakeProvider:

    def generate(self, prompt):
        return "AI stands for Artificial Intelligence."



# Test the LLM Client
class TestLLMClient(unittest.TestCase):

    def test_generate(self):
        provider = FakeProvider()

        client = LLMClient(provider)

        response = client.generate("What is AI?")

        self.assertEqual(
            response,
            "AI stands for Artificial Intelligence."
        )



# Run the test
if __name__ == "__main__":
    unittest.main()
```

7. Explanation of the Program

7.1 LLMProvider – common interface

LLMProvider is an abstract base class. Its abstract generate(prompt) method defines the operation that each provider class must implement.

7.2 GeminiProvider

GeminiProvider creates a Google Gen AI client using the API key. Its generate method sends the prompt to gemini-3.5-flash-lite and returns the generated text.

7.3 OllamaProvider

OllamaProvider sends a POST request to http://localhost:11434/api/generate. The request specifies the llama3.2 model, the prompt, and stream=False so the response is returned as one complete result. The generated text is read from the JSON field named response.

7.4 LLMClient

LLMClient receives a provider object. Its generate method forwards the prompt to that provider. The main program therefore uses the same client method for Gemini and Ollama.

7.5 FakeProvider and unit testing

FakeProvider returns a fixed sample sentence. assertEqual checks whether the actual response matches the expected response. This tests the common client without making an external API call.

8. Testing and Execution

8.1 Unit test

The test command is:

```
python -m unittest -v test_client.py
```

A successful run should report test_generate ... ok, followed by Ran 1 test and OK. Insert a screenshot of the actual test output.

8.2 Ollama execution

The program was started with:

```
python main.py
```

Option 2 was selected for Ollama. The prompt entered was:

```
What is Artificial Intelligence? Explain in simple words.
```

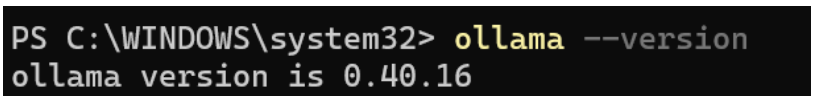
Ollama returned an explanation of AI.

8.3 Gemini execution

The program was run again using python main.py. Option 1 was selected for Gemini and the same sample prompt was entered. Gemini successfully returned a response. The Gemini provider uses gemini-3.5-flash-lite, the model that worked in the Week 1 setup.

9. Screenshots

OLLAMA VERSION



LLAMA 3.2 MODEL

```
PS C:\WINDOWS\system32> ollama pull llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100% ▕▕▕
```

```
[notice] A new release of pip is available: 24.0 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\ADMIN\OneDrive\Desktop\week-2> python -m unittest -v test_client.py
test_generate (test_client.TestLLMClient.test_generate) ... ok

----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

OLLAMA RESPONSE

```
PS C:\Users\ADMIN\OneDrive\Desktop\week-2> python main.py
Choose your AI Provider:
1. Gemini
2. Ollama
Enter your choice (1 or 2): 2

Enter your prompt: What is Artificial Intelligence? Explain in simple words.

AI Response:
Artificial Intelligence (AI) is like a super-smart computer that can think and learn like a human. It's like a robot that can do tasks on its own, without being told exactly what to do.

Imagine you have a really smart friend who can:

* Understand what you want to say
* Remember what you liked/disliked in the past
* Make decisions on its own
* Learn from its mistakes

That's basically what AI does, but instead of being a person, it's a computer program. It uses special algorithms and data to make decisions, learn from experience, and even talk to humans.

There are many types of AI, such as:

* Machine learning (which can learn from data)
* Natural Language Processing (which can understand and generate human language)
* Computer vision (which can recognize and interpret images)

The goal of AI is to make life easier and more convenient for humans, and to help us make better decisions.

Does that make sense?
PS C:\Users\ADMIN\OneDrive\Desktop\week-2>
```

GEMINI RESPONSE

```
PS C:\Users\ADMIN\OneDrive\Desktop\week-2>
PS C:\Users\ADMIN\OneDrive\Desktop\week-2> python main.py
Choose your AI Provider:
1. Gemini
2. Ollama
Enter your choice (1 or 2): 1

Enter your prompt: What is Artificial Intelligence? Explain in simple words.

AI Response:
In simple words, **Artificial Intelligence (AI)** is **making computers think and learn like humans.**

Normally, a computer only does what a human tells it to do, step by step. But with AI, we give computers the ability to figure things out, solve problems, and make decisions on their own.

Here are a few everyday examples to help you understand:

*   **Recommendation Systems:** When Netflix or YouTube suggests a movie or video you might like, that's AI watching what you usually watch and guessing your taste.
*   **Voice Assistants:** Siri and Alexa use AI to understand your spoken words and answer your questions.
*   **Video Games:** The computer-controlled characters you play against use AI to figure out how to fight or trick you.

**Think of it like teaching a child:**
Instead of giving a computer a strict rulebook for everything, we give it a lot of information (data) and let it learn from practice—just like how humans learn from experience.
```

10. Learning Outcomes

- Installed and verified Ollama on Windows.
- Downloaded and used the Llama 3.2 model locally.
- Connected a Python application to Google Gemini using the Google Gen AI SDK.
- Used an abstract base class to define a common interface for LLM providers.
- Implemented separate GeminiProvider and OllamaProvider classes.
- Used the same LLMClient.generate(prompt) method with either provider.
- Sent an HTTP POST request to the local Ollama API and read its JSON response.
- Created a fake provider and used Python unittest to test the common client.
- Stored the Gemini API key in .env and excluded it from Git using .gitignore.

11. Conclusion

The Week 2 Provider-Agnostic LLM Client was implemented in Python. It supports Google Gemini and Ollama with the Llama 3.2 model through one common LLMClient interface. The Ollama execution produced a local AI response, and the Gemini execution was also confirmed successful. A unit test was created to verify the common client's generate method using a fake provider. The project demonstrates how different LLM services can be accessed through a consistent application interface.